

LAB # 9

ENCAPSULATION, ABSTRACT CLASSES AND INTERFACES

OBJECTIVE:

- To understand concepts of encapsulation
- To understand the concept of Abstract class and Interfaces.

THEORY:

Data Encapsulation:

All C++ programs are composed of the following two fundamental elements:

- Program statements (code): This is the part of a program that performs actions, and they are called functions.
- Program data: The data is the information of the program which affected by the program functions.

Encapsulation is an Object-Oriented Programming concept that **binds together the data and functions** that manipulate the data, and that keeps both safe from outside interference and misuse. Data encapsulation led to the important OOP concept of data hiding.

Data encapsulation is a mechanism of bundling the data, and the functions that use them.

C++ supports the properties of encapsulation and data hiding through the creation of user-defined types, called classes. We already have studied that a class can contain private, protected and public members. By default, all items defined in a class are private.

```
class Rectangle {  
    public:  
        int length;  
        int breadth;  
  
        int getArea() {  
            return length * breadth;  
        }  
};
```

In the above program, the function getArea() calculates the area of a rectangle. To calculate the area, it needs length and breadth. Hence, the data members (length and breadth) and the function getArea() are kept together in the Rectangle class.

The variables length and breadth are private. This means that they can be accessed only by other members functions of the Box class, and not by any other part of your program. This is one way encapsulation is achieved.

To make parts of a class public (i.e., accessible to other parts of your program), you must declare them after the public keyword. All variables or functions defined after the public specifier are accessible by all other functions in your program.

Practice Example 1:

Encapsulating data members and functions

```
#include <iostream>
using namespace std;

class Passport {
private:
    string country_of_origin = "Pakistan";
public:
    int passport_number;
    string passport_name;
    void dothis() {
        cout << passport_name << "s " << country_of_origin << " passport number is " <<
passport_number;
    }
};

int main() {
    Passport mypassport;
    mypassport.passport_number = 123456;
    mypassport.passport_name = "Ahmed Raza";
    mypassport.dothis();
    return 0;
}
```

Data Abstraction:

Data abstraction refers to providing only essential information to the outside world and hiding their background details, i.e., to represent the needed information in program without presenting the details.

Abstract Classes

An abstract class is one that is not used to create objects. An abstract class is designed only to act as a base class.

Characteristics of Abstract Class

1. Abstract class provide an appropriate base class from which other classes can inherit.
2. Abstract classes cannot be used to instantiate objects and serves **only as an interface**.
3. Attempting to instantiate an object of an abstract class causes a compilation error.

A class is made abstract by declaring at least one of its functions as pure virtual function. A pure virtual function is specified by placing "= 0" in its declaration as follows:

Practice Example 2:

```
#include <iostream>
using namespace std;
class Demo{
public:
    virtual void f1()=0; // pure virtual function
};

int main(){
    Demo obj;
    obj.f1();
}
```

```
}
```

Why can't we create an Object of an Abstract Class?

When we create a pure virtual function in Abstract class, we reserve a slot for a function in the VTABLE, but doesn't put any address in that slot. Hence the VTABLE will be incomplete.

As the VTABLE for Abstract class is incomplete, hence the compiler will not let the creation of object for such class and will display an error message whenever you try to do so.

Practice Example 2:

```
#include <iostream>
using namespace std;

class Demo{
public:
    virtual void f1()=0; // pure virtual function

    void f2(){
        cout<<" In f2 Function" << endl;
    }
};

// Abc class inherited from Demo class which is abstract class
// Then Abc also contain the f1 and f2 functions, where f1 is pure virtual function
// Now Abc class also becomes abstract class
class Abc: public Demo{
};

int main(){
    Abc obj;
    obj.f2();
}
```

Practice Example 3:

Use Function overriding to solve the above issue:

```
#include <iostream>
using namespace std;

class Demo{
public:
    virtual void f1()=0; // pure virtual function

    void f2(){
        cout<<" In f2 Function" << endl;
    }
};

// here we apply function over riding
class Abc: public Demo{
public:
```

```

    void f1(){
        cout<<" I am f1 Function" << endl;           // Function overriding
    }
};

int main(){
    Abc obj;
    obj.f1();
    obj.f2();
}

```

Practice Example 4:

Consider the following example where base class (which is abstract) provides an interface to the derived class to implement a function called `getArea()`:

```

#include <iostream>
using namespace std;

// Base class
class Shape
{
public:
    // pure virtual function providing interface framework.
    virtual int getArea() = 0;

    void setWidth(int w)
    {
        width = w;
    }

    void setHeight(int h)
    {
        height = h;
    }
protected:
    int width;
    int height;
};

// Derived classes
class Rectangle: public Shape
{
public:
    int getArea()
    {
        return (width * height);
    }
};

// Derived classes
class Triangle: public Shape
{
public:

```

```

int getArea()
{
return (width * height)/2;
}
};

int main(void)
{
Rectangle Rect;
Triangle Tri;
Rect.setWidth(2);
Rect.setHeight(7);
    // Print the area of the object.
cout<< "Total Rectangle area: " <<Rect.getArea() <<endl;
Tri.setWidth(2);
Tri.setHeight(7);
    // Print the area of the object.
cout<< "Total Triangle area: " <<Tri.getArea() <<endl;
return 0;
}

```

You can see how an **abstract class defined an interface in terms of getArea() and two other classes implemented same function but with different algorithm to calculate the area specific to the shape.**

Interfaces:

An interface describes the behavior or capabilities of a C++ class without committing to a particular implementation of that class. C++ classes provide great level of data abstraction. They provide sufficient public methods to outside the world to play with the functionality of the objects and to manipulate object data i.e. state without actually knowing how class has been implemented internally.

LAB TASKS:

1. Write a program to find out the payroll system using encapsulation. Declare the base class emp. Define and declare the function get() to get the employee details (id, name, designation). Declare the derived class salary. Declare and define the function get1() to get the salary details (basic pay, home allowance, health allowance,tax). Define the function calculate() to find the net pay. Define the function display().

NOTE: net pay = basic pay + home allowance + health allowance – tax;

2. Modify the above program using abstract class concept.
3. Create a class named Person, which contains
 - i. A pure virtual function named print()
 - ii. Two data fields i.e. personName and age

LAB OUTCOMES:

- Student become familiar with what is data hiding.
- Student become familiar with the encapsulation and abstraction.

LAB # 10

POLYMORPHISM, COMPILE TIME POLYMORPHISM (FUNCTION OVERLOADING)

OBJECTIVE

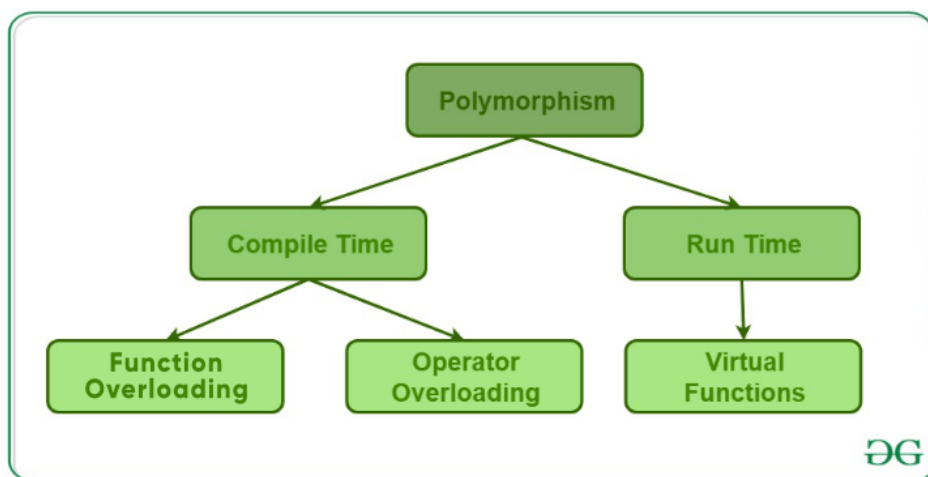
- To understand the concept of Polymorphism
- To understand the concept of Function overloading

THEORY

The word “polymorphism” means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. A real-life example of polymorphism is a person who at the same time can have different characteristics. A man at the same time is a father, a husband, and an employee. So the same person exhibits different behavior in different situations. This is called polymorphism. Polymorphism is considered one of the important features of Object-Oriented Programming.

Types of Polymorphism

- Compile-time Polymorphism
- Runtime Polymorphism



Types of Polymorphism

Function Call Binding with class Objects

Connecting the function call to the function body is called Binding. When it is done before the program is run, it's called Early Binding or Static Binding or Compile-time Binding.

1. Compile-Time Polymorphism

This type of polymorphism is achieved by function overloading or operator overloading.

A. Function Overloading

When there are multiple functions with the same name but different parameters, then the functions are said to be **overloaded**, hence this is known as Function Overloading. Functions can be overloaded by **changing the number of arguments** or/and **changing the type of arguments**. In simple terms, it is a feature of object-oriented programming providing many functions that have the same name but distinct parameters when numerous tasks are listed under one function name. There are certain Rules of Function Overloading that should be followed while overloading a function.

Below is the C++ program to show function overloading or compile-time polymorphism:

Practice Example 1:

// C++ program to demonstrate function overloading or Compile-time Polymorphism

```
#include <iostream>
using namespace std;

class Geeks {
public:
    // Function with 1 int parameter
    void func(int x)
    {
        cout << "value of x is " << x << endl;
    }

    // Function with same name but
    // 1 double parameter
    void func(double x)
    {
        cout << "value of x is " << x << endl;
    }

    // Function with same name and
    // 2 int parameters
    void func(int x, int y)
    {
        cout << "value of x and y is " << x << ", " << y
            << endl;
    }
};

// Driver code
int main()
{
    Geeks obj1;
    // Function being called depends on the parameters passed
    // func() is called with int value
    obj1.func(7);

    // func() is called with double value
    obj1.func(9.132);

    // func() is called with 2 int values
    obj1.func(85, 64);
    return 0;
}
```

Output

value of x is 7

value of x is 9.132

value of x and y is 85, 64

Explanation: In the above example, a single function named `func()` acts differently in three different situations, which is a property of polymorphism. To know more about this, you can refer to the article – Function Overloading in C++.

Practice Example 2:

Following is the example where same function **`print()`** is being used to print different data types:

```
#include <iostream>
using namespace std;
class printData
{
    public:
    void print(int i) {
        cout << "Printing int: " << i << endl;
    }

    void print(double f) {
        cout << "Printing float: " << f << endl;
    }
};

int main(void)
{
    printData pd;
    pd.print(5);
    pd.print(500.263);
}
```

Explanation:

Another appropriate example would be a *Print (object O)* method. In this case one might like the method to be different when printing, for example, text or pictures. The two different methods may be overloaded as *Print (text_object T)*; *Print (image_object P)*. If we write the overloaded print methods for all objects our program will “print”, we never have to worry about the type of the object, and the correct function call again, the call is always: *Print(something)*.

Practice Example 3:

Write a c++ program to implement Function overloading

```
#include <iostream>
#include <conio.h>
using namespace std;

class calculatingvolume
{
    public:
    int volume(int a)    //for cube
    {
        return a*a*a;
    }

    int volume(int a, int b, int c)    //for cuboid
    {
        return a*b*c;
    }
}
```



```

}

int volume (int r, int h)    //for cylinder
{
return 3.14*r*r*h;
}
};

int main()
{
calculatingvolume c;
cout<<"volume of the cube is:"<<endl;
cout<<c.volume(10)<<endl; cout<<"volume of the cuboid is:"<<endl;
cout<<c.volume(5,10,15)<<endl; cout<<"volume of the cylinder is:"<<endl;
cout<<c.volume(10,15)<<endl;
getch(); return 0;
}

```



```

C:\c++\functionoverloading.exe
volume of the cube is:
1000
volume of the cuboid is:
6000
volume of the cylinder is:
6280

```

Output of function overloading program

LAB TASKS:

1. Write down the program using function overloading concepts, at least four function should be overloaded.
2. Consider a class named **Calculator** with typical four specific functionalities i.e. **addition**, **subtraction**, **multiplication**, and **division**. Implement these functionalities as four functions with two parameters. It is also required to overload all these functions for **int** and **double** data types. In the **main** function, create an object of class **Calculator** and invoke its member functions while passing parameters of **int** and **double** type.

LAB OUTCOMES:

- Student become familiar with the usage of compile time polymorphism
- Student become familiar with the usage of function overloading

LAB # 11

COMPILE TIME POLYMORPHISM (OPERATOR OVERLOADING)

OBJECTIVE

- To become familiar with the concept of operator overloading
- To be able to use this technique in programming

THEORY

OPERATOR OVERLOADING:

C++ has the ability to provide the operators with a special meaning for a data type, this ability is known as operator overloading. For example, we can make use of the addition operator (+) for string class to concatenate two strings. We know that the task of this operator is to add two operands. So a single operator '+', when placed between integer operands, adds them and when placed between string operands, concatenates them.

Operator overloading allows the programmer to define how operators (such as +, -, ==, =, and !) should interact with various data types. Because operators in C++ are implemented as functions, operator overloading works very analogously to function overloading.

Operators are overloaded by writing a function definition as you normally would, except that the function name now becomes the keyword **operator** followed by the symbol for the operator being overloaded. The following operators cannot be overloaded:

`., .* , ::, ?:, #`

When you call an overloaded **operator**, the compiler determines the most appropriate definition to use by comparing the argument types you used to call the function or operator with the parameter types specified in the definitions.

Practice Example 1:

OVERLOADING UNARY OPERATOR

Unary operators act on only one operand. Examples of unary operators are ++, --. These operators do not work directly with the object. Every time you want to increment any object you need to call a member function defined for that purpose. But the listing would be more readable if we could have used the increment operator ++ instead of calling function. The program given below demonstrates how increment operator is overloaded and used with objects.

// Overload ++ increment operator when used as a postfix

```
#include<iostream>
using namespace std;
```

```
int main()
{
int a = 3;
int c;
c = a++;
cout<<"Result 1- Value of c is : " << c << endl ;
cout<<"Result 2- Value of a is : " << a << endl ;
return 0;
}
```

Practice Example 2:

// Overload ++ increment operator when used as a prefix

```
#include<iostream>
using namespace std;

int main()
{
int a = 3;
int c;
c = ++a;
cout<<"Result 1- Value of c is : " << c << endl ;
cout<<"Result 2- Value of a is : " << a << endl ;
return 0;
}
```

Practice Example 3:**OVERLOADING BINARY OPERATORS:**

Like unary operators binary operators can also be overloaded. If you want to perform any binary operation like add, subtract, divide, multiply and compare two objects you need to call member functions written to perform these operations. By overloading binary operators we can reduce our programming code and simplify our statement. The example program demonstrates how arithmetic, comparison, and arithmetic assignment operators are overloaded.

Program:

```
#include <iostream>
using namespace std;

class Time {
private:
int hour;
int minute;

public:
Time() : hour(0), minute(0) { }

void in() {
cout << "Enter the time: ";
cin >> hour;
cin >> minute;
}

// Overload the + operator
Time operator + (const Time & obj) {
Time temp;
temp.hour = hour + obj.hour;
temp.minute = minute + obj.minute;
if (temp.minute>=60)
{
temp.hour+=1;
temp.minute-=60;
}
}
```

```

        if (temp.hour>24)
            temp.hour=1;
        return temp;
    }

    void out() {
        cout<<"Time is "<< hour<<"hrs "<<minute<<"min";
    }
};

int main() {
    Time T1, T2, result;

    cout << "Enter first time in hours and minutes one by one :n";
    T1.in();

    cout << "Enter second time in hours and minutes one by one :n";
    T2.in();

    // T1 calls the operator function
    // T2 is passed as an argument to the function
    result = T1 + T2;
    result.out();

    return 0;
}

```

OUTPUT:

```

Enter first time in hours and minutes one by one:
Enter the time:11
56
Enter second time in hours and minutes one by one:
Enter the time: 2
10
Time is 14hrs 6 min

```

EXPLANATION:

This code demonstrates binary Operator overloading in C++. The Time class represents time in hours and minutes. The + Operator is overloaded to add two Time objects together.

Upon execution, the program prompts the user to input two times, adds them using the overloaded + Operator, and displays the result. In the provided output, 11hrs 56min and 2hrs 10min are added together, resulting in 14hrs 6min.

Practice Example 3:

Relational Operators Overloading

There are various relational operators supported by C++ language like (<, >, <=, >=, ==, etc.) which can be used to compare C++ built-in data types.

You can overload any of these operators, which can be used to compare the objects of a class.

```

#include <iostream>
using namespace std;

class Student{
    int feet = 0; //can be b/w 0 & infinity
    int inches = 0; // can be b/w 0 & 12

    public:
    void getHeight(int f, int i){
        feet = f;
        inches = i;
    }

    // const and & added check explanation above(Before code) why
    bool operator > (const Student& s2)
    {
        if(feet > s2.feet)
            return true;

        else if(feet == s2.feet && inches > s2.inches)
            return true;

        return false;
    }
};

int main()
{
    Student s1,s2;

    s1.getHeight(5,10);
    s2.getHeight(6,1);

    if(s1 > s2)
        cout << "Student 1 is taller" << endl; else if(s2 > s1)
        cout << "Student 2 is taller" << endl;
    else
        cout << "Both have equal height" << endl;

    return 0;
}

```

Output:

Student 2 is taller

Practice Example 4:

Assignment Operator Overloading

The assignment operator,"=", is the operator used for Assignment. It copies the right value into the left value. Assignment Operators are predefined to operate only on built-in Data types.

Overloading assignment operator in C++ copies all values of one object to another object.

// C++ program to check if two clocks are showing the same time or not

```
#include <iostream>
using namespace std;

class Time
{
    int hour, mins, secs;
public:
    Time(){
        hour=0, mins=0, secs=0;
    }

    // parameterized constructor
    Time(int h, int m, int s)
    {
        if(h>=0 && h <=23 && m >=0 && m <=59 && s >0 && s <=59)
            { hour=h, mins=m, secs=s; }
        else
            cout << "Invalid time format, values would be 00:00:00 by default" << endl;
    }

    // returns true if both times are same
    bool operator == (const Time& t2)
    {
        return (hour == t2.hour
            && mins == t2.mins
            && secs == t2.secs);
    }
};

int main()
{
    Time t1(7,11,30);
    Time t2(3,30,41);

    if(t1 == t2)
        cout << "Clocks show same time";
    else
        cout << "Clocks show different times";

    return 0;
}
```

Output

Clocks show different times

LAB TASKS:

1. Let us multiply two fractions using the overloading of the multiplication operator in C++.
2. A C++ program to overload a prefix decrement operator
3. Adding the dimension of two Cube objects and printing the results for volume for the resultant cube by using binary operator overloading
Cube 3 = Cube1 + Cube2

LAB OUTCOMES:

- Student become familiar with the usage of compile time polymorphism
- Student become familiar with the usage of operator overloading